Add vs Update vs Attach vs Entry.State

Visual Lesson Summary • Entity states and the SQL they lead to

1. Core Idea

These methods mainly tell EF Core how to track an entity by setting its **EntityState**. They do **not execute SQL directly**. SQL is generated and executed later when **SaveChanges()** is called.

SaveChanges() converts the tracked state into database commands

2. The Four Methods

Add() — New row

Use when the entity is brand new. EF Core marks it as **Added**, which leads to an INSERT when saved.

```
var student = new Student
{
        FirstName = "Ali",
        LastName = "Hassan",
        Email = "ali@test.com"
};

context.Students.Add(student);
context.SaveChanges();
```

Update() — Broad update

Use when an existing entity should be updated. EF Core marks the entity as **Modified**. Be careful: many properties may be marked modified, so an incomplete object can overwrite stored values with defaults or nulls.

```
var student = new Student
{
        StudentId = 1,
        FirstName = "Ali",
        PhoneNumber = "0799999999"
};

context.Students.Update(student);
context.SaveChanges();
```

Attach() — Start tracking

Use when the row already exists and you want EF Core to start tracking it without marking it modified. The initial state is **Unchanged**; no UPDATE is sent unless a property is changed and detected.

```
var student = new Student
{
        StudentId = 1
};

context.Attach(student);

student.PhoneNumber = "0798888888";
context.SaveChanges();
```

Entry(...).State — Manual state control

Use this when you want to choose the entity state explicitly, such as Added, Deleted, Unchanged, or Modified.

```
context.Entry(student).State =
        EntityState.Modified;

// Other possible states:
EntityState.Added;
EntityState.Deleted;
EntityState.Unchanged;
```

3. Example — Delete Without Loading the Row

```
var student = new Student
{
        StudentId = 1
};

context.Entry(student).State =
        EntityState.Deleted;

context.SaveChanges();
```

This tells EF Core to treat the entity as Deleted. The lesson's expected SQL shape is a DELETE using StudentId as the key.

4. Compare the Methods

Method	Typical state	Effect on SaveChanges()	Use when
Add()	Added	INSERT	Creating a new row
Update()	Modified	UPDATE	Updating a complete existing object; use carefully
Attach()	Unchanged	No SQL until changes are made	Start tracking an existing row
Entry.State	Chosen manually	Depends on selected state	Advanced, explicit state control

5. Common Mistakes

• Using Add() for an existing row may attempt a duplicate INSERT.

• Using Update() with a partial DTO may overwrite fields that were not supplied.

• Calling Attach() and expecting an UPDATE immediately — Attach() initially sets the entity to Unchanged.

• Setting EntityState manually without understanding the resulting operation can cause unintended INSERT, UPDATE, or DELETE commands.

6. Professional Decision Guide

New row? Use Add().
Complete existing object to update? Update() may fit, but check every field.
Existing row, start tracking without changes? Attach().
Need explicit state behavior? Entry(entity).State.

> **Mental anchor:** These methods mainly choose the entity state. The state determines what SaveChanges() will ask the database to do.